CONTENTS

TABLE OF THE CONTENTS		
CHAPTER	**TITLE**	**PAGE.NO**
I	**INTRODUCTION**	1
	1.1 AN OVERVIEW	2
	1.2 OBJECTIVES OF THE PROJECT	3
	1.3 ORGANIZATION PROFILE	4
	1.4 SCOPE OF THE SYSTEM	5
II	**SYSTEM ANALYSIS**	6
	2.1 EXISTING SYSTEM	6
	2.1.1 DRAWBACKS OF EXISTING SYSTEM	7
	2.2 PROPOSED SYSTEM	8
	2.2.1 FEATURES OF PROPOSED SYSTEM	9
	2.3 SOFTWARE SPECIFICATION	10
	2.4 HARDWARE SPECIFICATION	11
	2.5 SOFTWARE ENVIRONMENT	12
III	**SYSTEM DESIGN AND DEVELOPMENT**	13
	3.1 ARCHITECTURAL DESIGN	13
	3.2 USECASE DIAGRAM	14
	3.3 STATE DIAGRAM	14
	3.4 SEQUENCE DIAGRAM	15
	3.5 DESIGN PROCESS	15
	3.5.1 DATABASE DESIGN	16
	3.5.2 INPUT DESIGN	17
	3.5.3 OUTPUT DESIGN	18
	3.6 TECHNIQUES USED	19
IV	**TESTING AND IMPLEMENTATION**	21
	4.1 SYSTEM TESTING	21
	4.2 SYSTEM IMPLEMENTATION	22
	4.3 SYSTEM MAINTENANCE	23
V	**CONCLUSION**	24
VI	**SCOPE OF THE FUTURE DEVELOPMENT**	25
BIBLIOGRAPHY		
APPENDICES		
(A) SOURCE CODE (B) SCREEN-SHOTS (C) TABLES (D) REPORTS		

I. INTRODUCTION

The rapid advancement of Artificial Intelligence and web-based technologies has significantly transformed traditional workflows across multiple industries, including architecture and construction. Building floor plan design, which has historically depended on manual drafting and professional expertise, remains a time-intensive and iterative process. Architects traditionally interpret client requirements, convert them into sketches, refine layouts through multiple revisions, and finally prepare technical drawings using CAD or BIM software. While these tools improve drafting efficiency, they do not eliminate the need for continuous human intervention, decision-making, and optimization during the conceptual planning stage. In modern construction environments where speed, cost-efficiency, and accessibility are critical, there is an increasing demand for intelligent systems that can automate early-stage architectural planning. Many individuals and small-scale builders require preliminary layouts before consulting professional architects, but existing tools either require technical expertise or rely on rigid templates that limit customization. This gap highlights the need for an intelligent, requirement-driven, and automated building plan generation system.

The proposed system, PlanAI, addresses this challenge by introducing an AI-powered framework capable of transforming user-defined requirements into structured, optimized, and standards-aware floor plans. By integrating generative artificial intelligence, rule-based architectural validation, spatial optimization algorithms, and real-time visualization technologies, the system automates the conceptual design workflow while maintaining architectural feasibility. Users can input parameters such as plot dimensions, number of rooms, number of floors, and design preferences, and the system generates both 2D blueprint-style plans and interactive 3D visualizations. The introduction of AI into architectural planning not only reduces time and cost but also improves consistency, scalability, and accessibility. The system demonstrates how modern generative models combined with deterministic constraint validation can create practical and professional layouts without replacing architects, but rather supporting and accelerating their work. This project therefore represents a convergence of Artificial Intelligence, web engineering, spatial computation, and cloud infrastructure to modernize the building plan generation process.

1.1 AN OVERVIEW

PlanAI is an Artificial Intelligence–driven web application developed to automate the generation of building floor plans based on user-defined requirements. The system is designed to simplify the early stages of architectural planning by converting structured input parameters into optimized, regulation-aware, and spatially efficient layouts. Unlike traditional drafting tools that require manual drawing and expert supervision, PlanAI follows an intelligent generation pipeline that integrates requirement analysis, AI-based layout synthesis, constraint validation, optimization, and visualization within a unified workflow. The system operates through a requirement-driven architecture. Users provide essential planning inputs such as plot dimensions, number of floors, room requirements, and design preferences through a structured interface. These inputs are processed by a backend engine that invokes a generative AI model to produce a structured spatial layout in machine-readable format. Instead of generating only graphical outputs, the system produces detailed layout data including room coordinates, dimensions, adjacency relationships, and floor mapping. This structured approach enables deterministic validation and optimization before rendering.

A rule-based validation module ensures that the generated layouts comply with architectural principles such as minimum room sizes, proper circulation space, logical adjacency between functional areas, and boundary constraints. An optimization module further evaluates space utilization efficiency and adjusts partitions to minimize dead space and improve overall functionality. Once validated and optimized, the layout is rendered into a professional 2D blueprint-style floor plan and an interactive 3D model for visualization. The platform is built using modern web technologies, including a React and TypeScript frontend for interactive user experience, cloud-based backend services for data storage and authentication, and serverless functions for AI orchestration. Real-time 3D visualization is implemented using browser-based rendering technologies, allowing users to inspect spatial arrangements dynamically. Overall, PlanAI provides a scalable, automated, and intelligent alternative to manual conceptual planning. It bridges the gap between user requirements and architectural design through AI-driven layout synthesis while maintaining professional structural logic and usability standards.

1.2 OBJECTIVES OF THE PROJECT

The primary objective of the PlanAI system is to develop an intelligent, automated, and scalable platform capable of generating building floor plans from structured user requirements using Artificial Intelligence and modern web technologies. The project aims to transform the traditional manual and time-consuming architectural planning process into a fast, optimized, and requirement-driven workflow. One of the key objectives is to automate the conceptual design phase of building planning. The system is intended to reduce dependency on manual drafting by generating spatial layouts automatically based on plot dimensions, number of rooms, floor count, and design preferences. By integrating generative AI models, the project seeks to enable intelligent room placement and layout synthesis without requiring users to possess professional architectural knowledge. Another objective is to ensure that the generated plans are not only fast but also practically feasible. The system incorporates rule-based architectural validation mechanisms to enforce constraints such as minimum room dimensions, proper circulation paths, logical adjacency between functional spaces, and boundary compliance.

This ensures that the output layouts maintain professional planning standards rather than producing arbitrary designs. The project also aims to optimize space utilization. Through computational evaluation and gap detection techniques, the system maximizes usable plot coverage while minimizing wasted or unusable areas. This objective focuses on improving functional zoning and spatial efficiency in automatically generated designs. A further objective is to provide dual-format visualization for better interpretation. The system generates both technical 2D blueprint-style plans and interactive 3D models, allowing users to analyze layouts from both engineering and visual perspectives. This enhances usability for both technical professionals and non-expert users. Scalability and accessibility are also central objectives of the project. By implementing a cloud-based architecture with serverless backend processing, the system ensures that multiple users can generate and manage plans simultaneously without requiring specialized hardware or software installation. Finally, the project aims to establish a foundation for future intelligent design systems by integrating generative AI, deterministic validation, optimization algorithms, and real-time visualization into a unified architectural planning framework.

1.3 ORGANIZATION PROFILE

Byt4ge is a technology startup engaged in the development of innovative digital products and customized software solutions for clients. The organization undertakes both in-house product development and client-based projects, focusing on delivering practical, scalable, and secure technology systems that address real-world business requirements.

The company provides end-to-end software development services, including full-stack web application development, enterprise software solutions, cloud-based platforms, and database architecture. Byt4ge designs responsive and user-friendly interfaces while building strong backend systems that ensure performance, security, and reliability. Our services also include API integration, real-time data management, authentication and authorization systems, and secure data handling practices.

In addition to core software services, Byt4ge develops automation tools and AI-driven applications that improve operational efficiency and support data-driven decision-making. The organization follows a structured development lifecycle that includes requirement analysis, system design, coding, testing, deployment, and maintenance.

Through continuous innovation and adherence to professional standards, Byt4ge aims to deliver high-quality digital solutions while providing interns and team members with hands-on experience in modern technologies and industry-oriented development practices.

1.4 SCOPE OF THE SYSTEM

The scope of the proposed system PlanAI encompasses the automation of conceptual building floor plan generation using Artificial Intelligence, structured rule validation, and real-time visualization technologies. The system is primarily focused on assisting users in generating optimized residential and small-scale commercial building layouts based on clearly defined requirements such as plot dimensions, number of rooms, number of floors, and design preferences. The system is designed to operate at the conceptual planning stage rather than the final structural engineering phase. It provides intelligently generated spatial arrangements that comply with fundamental architectural standards including room sizing, adjacency logic, circulation pathways, and space utilization efficiency.

The scope includes automatic generation of structured layout data, validation of dimensional constraints, and visualization in both 2D blueprint and interactive 3D formats. Users can generate, review, modify basic inputs, and export preliminary plans for further professional refinement. The platform supports multi-floor planning, structured room categorization, and optimization of plot coverage. It includes cloud-based storage capabilities that allow authenticated users to save and retrieve generated layouts. The scope also covers scalable web deployment, enabling multiple users to access and generate plans simultaneously without local installation requirements. However, the system does not replace licensed architects or structural engineers.

It does not perform detailed structural load calculations, foundation design, electrical wiring diagrams, plumbing layouts, or legal municipal approval documentation. These advanced engineering and regulatory processes remain outside the current scope and are intended for professional evaluation following conceptual plan generation. In summary, the scope of PlanAI is confined to intelligent conceptual layout generation, optimization, validation, and visualization. It serves as a decision-support and rapid prototyping tool for building planning, forming a bridge between initial user requirements and detailed professional architectural development.

II. SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The existing system for building floor plan creation primarily relies on manual architectural drafting supported by computer-aided design tools and parametric modeling software. In current practice, the design process begins with client consultation, where requirements such as plot size, number of rooms, intended building purpose, budget, and aesthetic preferences are collected. Based on these discussions, architects manually conceptualize and sketch layout ideas before converting them into detailed technical drawings using CAD or BIM software platforms. Software such as AutoCAD, Revit, ArchiCAD, and SketchUp are widely used in the industry to produce architectural drawings and 3D models. These tools provide precision drafting capabilities, structural modeling, and documentation features. However, they function primarily as design assistance tools rather than autonomous layout generators.

Every wall, partition, opening, and dimension must be manually defined and adjusted by the architect. The creative and logical reasoning required for room placement, zoning, circulation flow, and space optimization remains dependent on human expertise. The existing workflow is iterative in nature. After preparing an initial draft, architects present the design to clients for review. Modifications such as resizing rooms, relocating staircases, adjusting circulation paths, or altering spatial hierarchy require manual redrawing and recalculation. Multiple revision cycles are common before final approval is achieved. This process increases both time consumption and cost. Some modern design tools incorporate parametric or template-based planning features. These systems allow users to adjust predefined templates or manipulate rule-based design parameters.

However, such systems still require professional knowledge to configure constraints properly. They do not intelligently interpret user requirements in a fully automated manner and are limited in flexibility when handling unconventional plot shapes or customized spatial demands. Research-level AI-based layout generators exist but are not widely integrated into commercial workflows. These systems often generate visual outputs without complete architectural validation, making them unsuitable for practical implementation without further manual refinement. Overall, the existing system is tool-assisted but not intelligence-driven. It supports drafting efficiency but does not eliminate the need for continuous human decision-making, spatial reasoning, and optimization.

2.1.1 DRAWBACKS OF EXISTING SYSTEM

The existing system of building floor plan design, although supported by advanced CAD and BIM tools, presents several limitations that affect efficiency, accessibility, and scalability. One of the primary drawbacks is the heavy dependence on skilled architects and designers. The conceptualization, spatial reasoning, and decision-making processes are entirely human-driven, making the workflow highly expertise-dependent. Non-technical users cannot independently generate or modify building layouts without professional assistance.

Another major drawback is the time-consuming iterative design process. Each modification requested by the client requires manual redrawing, recalculation of dimensions, and adjustment of spatial relationships. This repeated cycle of drafting and revision increases project timelines significantly, especially during the conceptual phase. Cost is also a significant limitation. Professional design services, drafting tools, and BIM software licenses involve substantial expenses. For small-scale projects or preliminary planning, the financial investment may not be justified, particularly when multiple draft variations are required before finalization.

Existing systems also lack intelligent automation. Template-based and parametric tools provide limited flexibility and cannot fully interpret diverse user requirements. They rely on predefined design patterns and manual parameter tuning rather than adaptive learning or requirement-driven generation. As a result, customization beyond standard templates can become complex and labor-intensive. Furthermore, most traditional tools do not automatically optimize space utilization. The efficiency of the layout depends on the architect's experience rather than computational evaluation. Dead spaces, inefficient circulation, and inconsistent zoning may occur if not carefully analyzed.

Another limitation is the absence of integrated real-time validation during conceptual generation. While professional software can check structural compliance, early-stage conceptual plans may not automatically enforce spatial standards such as minimum room dimensions, proper ventilation logic, or functional adjacency rules. Finally, existing systems do not provide instant dual-mode visualization for rapid decision-making. Generating both technical blueprints and interactive 3D models often requires separate processes and additional effort.

2.2 PROPOSED SYSTEM

The proposed system, PlanAI, is an Artificial Intelligence-driven building floor plan generation platform designed to automate the conceptual architectural planning process. Unlike the existing manual and template-based systems, the proposed solution follows a requirement-driven, intelligent workflow that transforms structured user inputs into optimized and validated building layouts. The system begins by collecting essential parameters such as plot dimensions, number of floors, room requirements, and design preferences through a guided web interface. These inputs are processed by a requirement analysis module that converts user selections into structured design constraints. This structured data acts as the conditioning input for the AI layout generation engine.

At the core of the proposed system is a generative AI model that produces structured spatial layout data instead of simple visual sketches. The model is prompted to generate machine-readable output containing room coordinates, dimensions, adjacency relationships, and floor mapping. This structured approach ensures that the generated plan can be evaluated and refined through deterministic algorithms rather than relying solely on visual output. Following layout generation, a rule-based validation module checks the design against architectural constraints such as minimum room size requirements, proper circulation width, logical room connectivity, and boundary compliance. Any violations are automatically corrected or adjusted before final output generation. An optimization module further enhances space utilization by minimizing unused regions and improving zoning efficiency.

The validated layout is then rendered into a professional 2D blueprint-style floor plan and an interactive 3D visualization model. Users can review the generated plan, compare alternatives, save their designs, and export them for further professional development. The proposed system integrates modern web technologies for frontend interaction, cloud-based backend services for storage and processing, serverless functions for AI orchestration, and real-time rendering technologies for visualization. This combination ensures scalability, accessibility, and high performance.

2.2.1 FEATURES OF PROPOSED SYSTEM

The proposed system PlanAI incorporates a comprehensive set of intelligent and technical features designed to automate, validate, and optimize building floor plan generation. These features collectively ensure accuracy, efficiency, usability, and scalability. One of the primary features of the system is AI-driven layout generation. The system uses a generative artificial intelligence model to synthesize structured floor plan layouts based on user-defined requirements. Unlike template-based tools, it dynamically arranges rooms according to spatial logic and learned architectural patterns. Another important feature is **s**tructured output generation. The AI produces layout data in a machine-readable format containing room dimensions, coordinates, adjacency relationships, and floor mapping. This structured format enables deterministic validation and optimization rather than relying only on graphical outputs. The system includes a rule-based architectural validation module.

This feature ensures compliance with fundamental planning standards such as minimum room dimensions, proper circulation width, logical room adjacency, and boundary alignment. It reduces the risk of impractical or infeasible layouts. An integrated **s**pace optimization engine evaluates layout efficiency and maximizes plot utilization. It identifies unused or fragmented spaces and adjusts room partitions to improve functional zoning and circulation flow. The platform provides dual-mode visualization, including professional 2D blueprint-style rendering and interactive 3D visualization. The 2D output includes wall hierarchies and spatial labeling, while the 3D viewer allows real-time inspection of spatial arrangements. The system supports multi-floor building planning, enabling users to generate layouts for buildings with more than one level. Floor-wise mapping ensures consistency and structured stacking logic. A **c**loud-based architecture ensures scalability and remote accessibility. Users can generate, save, and retrieve plans through secure authentication mechanisms without requiring specialized local installations. The system also includes PDF export functionality, allowing users to download blueprint-style plans for sharing or further professional refinement.

2.3 SOFTWARE SPECIFICATION

The software specification of the proposed system PlanAI defines the development technologies, frameworks, tools, and runtime environments used to design, implement, and deploy the application. The system is developed using a modern full-stack architecture that integrates frontend technologies, backend services, AI processing, and visualization libraries to ensure performance, scalability, and maintainability.

Frontend Technologies:

The user interface of the system is developed using React as the primary frontend framework. React enables component-based architecture, efficient state management, and dynamic rendering of user interfaces. The application is written in TypeScript, which enhances code reliability through static type checking and improves maintainability in large-scale applications. Styling is implemented using Tailwind CSS, a utility-first CSS framework that allows responsive and consistent design. UI components are structured using modern component libraries to ensure accessibility and usability.

Backend Technologies:

The backend infrastructure is built using a cloud-based platform such as Supabase, which provides database management, authentication services, and serverless edge functions. The backend handles request processing, AI orchestration, structured data validation, and persistent storage of generated plans. Serverless functions are used to manage AI model invocation and rule-based validation processes efficiently.

LLM Integration:

The system integrates a Large Language Model through an API gateway. The LLM is configured with structured prompting techniques to generate machine-readable layout data in JSON format. This integration enables requirement-driven layout generation instead of simple textual or graphical output. The structured output allows further validation, optimization, and rendering within the system workflow.

Visualization Libraries:

For 2D rendering, scalable vector graphics and canvas-based rendering techniques are used to generate blueprint-style floor plans. For 3D visualization, the system uses Three.js with React Three Fiber, enabling real-time interactive rendering directly within the web browser.

Supporting Tools:

- Vite as the build and development tool for fast bundling and hot module replacement.
- TanStack React Query for asynchronous state management and API request handling.
- React Router for navigation between application modules.
- jsPDF or equivalent library for exporting floor plans in PDF format.

Operating Environment:

The application runs in modern web browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox. Since it is web-based, no specialized installation is required on the client side.

2.4 HARDWARE SPECIFICATION

The hardware specification defines the minimum and recommended system requirements necessary to develop, deploy, and operate the proposed system efficiently. Since PlanAI is a web-based application with cloud backend integration, the hardware requirements are moderate for end users and slightly higher for development and server environments. For development purposes, a computer system with at least an Intel Core i5 processor or equivalent is recommended.

A minimum of 8 GB RAM is required to handle frontend development tools, backend services, and AI integration processes smoothly. Storage space of at least 256 GB is recommended to accommodate development tools, local dependencies, and testing datasets. A stable internet connection is essential for cloud integration, API communication, and deployment. For server-side infrastructure, the system relies on cloud-based serverless platforms. The cloud provider manages computational resources dynamically, ensuring scalability and high availability.

Since AI processing is handled via external AI APIs, high-performance GPU hardware is not mandatory on the local system. For end users, the hardware requirement is minimal. Any modern computer or laptop with at least 4 GB RAM and a stable internet connection is sufficient. The system is browser-based and compatible with standard input devices such as keyboard and mouse. No specialized hardware components are required for operation.

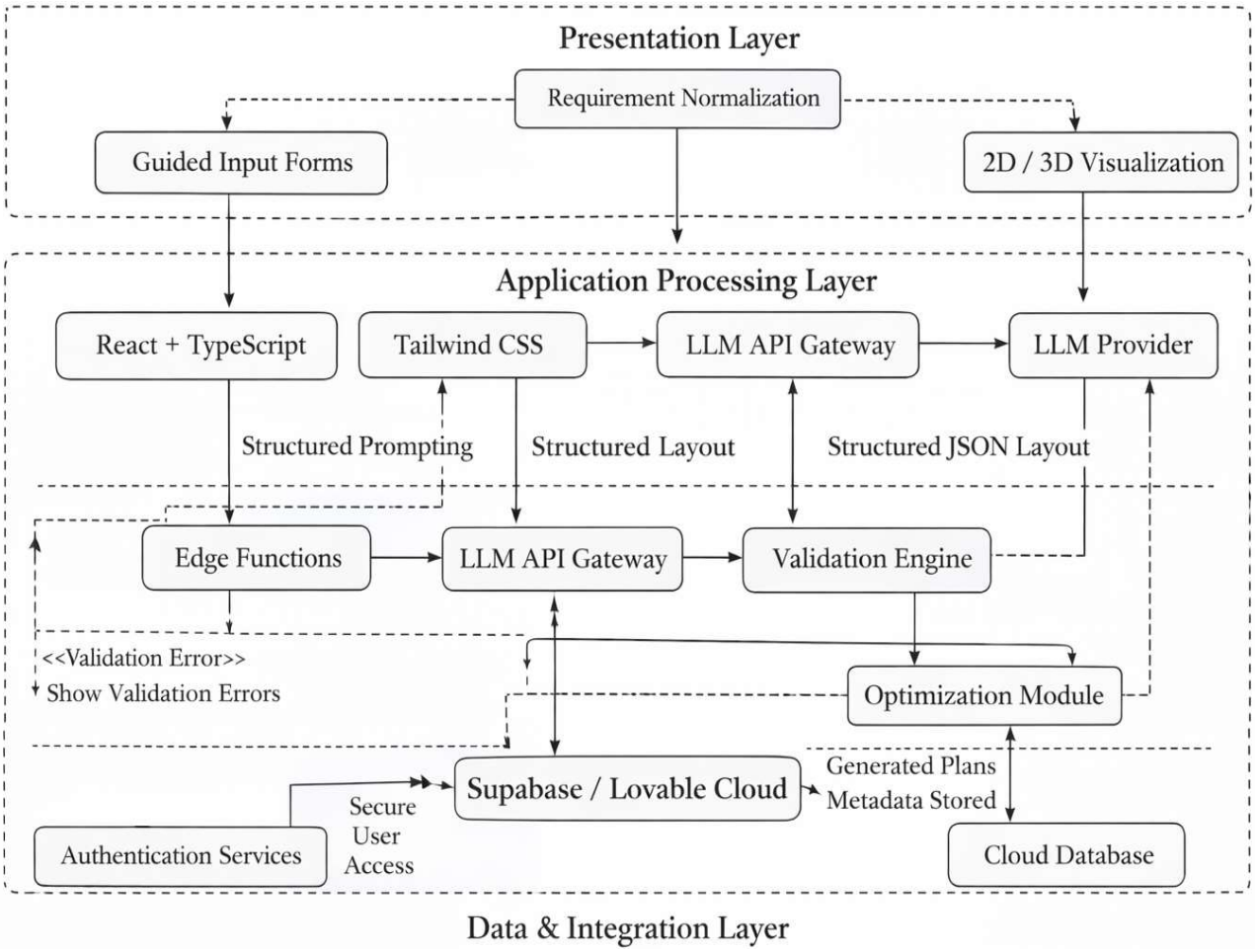
2.5 SOFTWARE ENVIRONMENT

The software environment of the proposed system PlanAI defines the platform setup, runtime ecosystem, development tools, and deployment configuration required for system execution. The application is designed to operate within a modern web-based environment supported by cloud infrastructure and API-driven integrations. The development environment includes Node.js as the runtime for managing frontend dependencies and build processes. The project uses Vite as the development server and build tool to enable fast compilation and efficient module bundling. The source code is managed using a version control system such as Git to ensure collaborative development and change tracking. The application is deployed as a web-based platform accessible through modern browsers including Google Chrome, Microsoft Edge, and Mozilla Firefox.

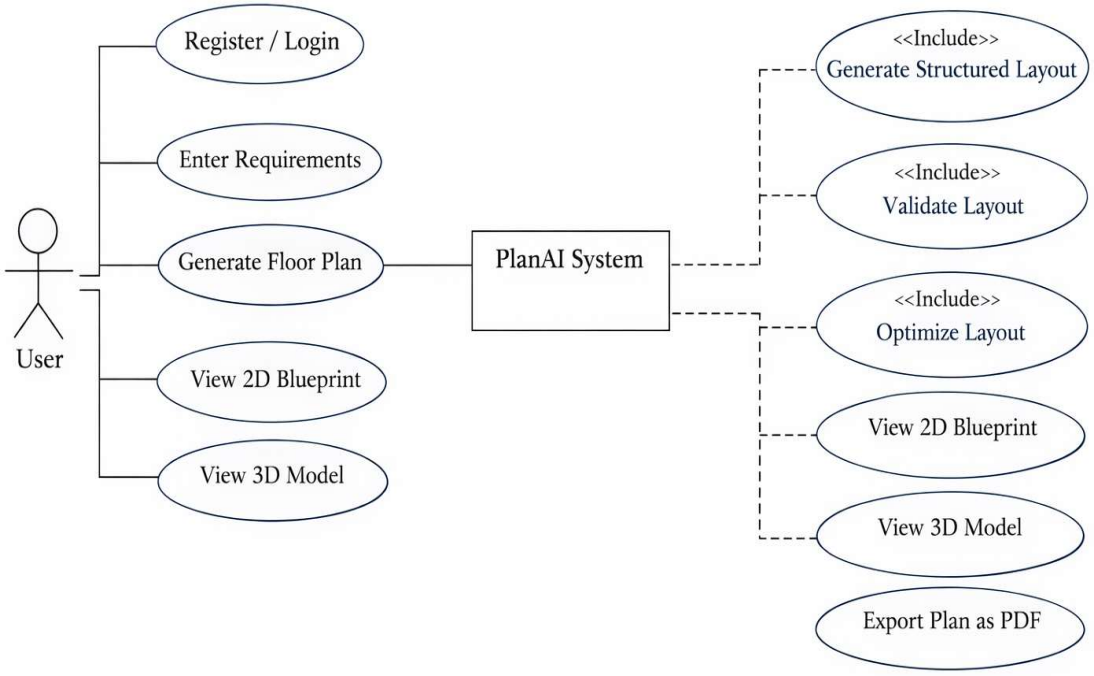
Since the system operates entirely through the browser, no additional client-side software installation is required. The frontend is executed in the browser environment using JavaScript and TypeScript, while backend operations are executed on cloud infrastructure. The backend environment is hosted on a cloud-based platform such as Supabase or Lovable Cloud, which provides managed database services, authentication mechanisms, and serverless edge functions. These serverless functions execute backend logic, handle LLM integration, validate structured layout data, and return processed outputs to the frontend. The system communicates with the integrated Large Language Model through secure API endpoints. Internet connectivity is required for AI invocation, data storage, authentication, and rendering services. Security configurations include HTTPS communication, token-based authentication, and secure database access control.

III. SYSTEM DESIGN AND DEVELOPMENT

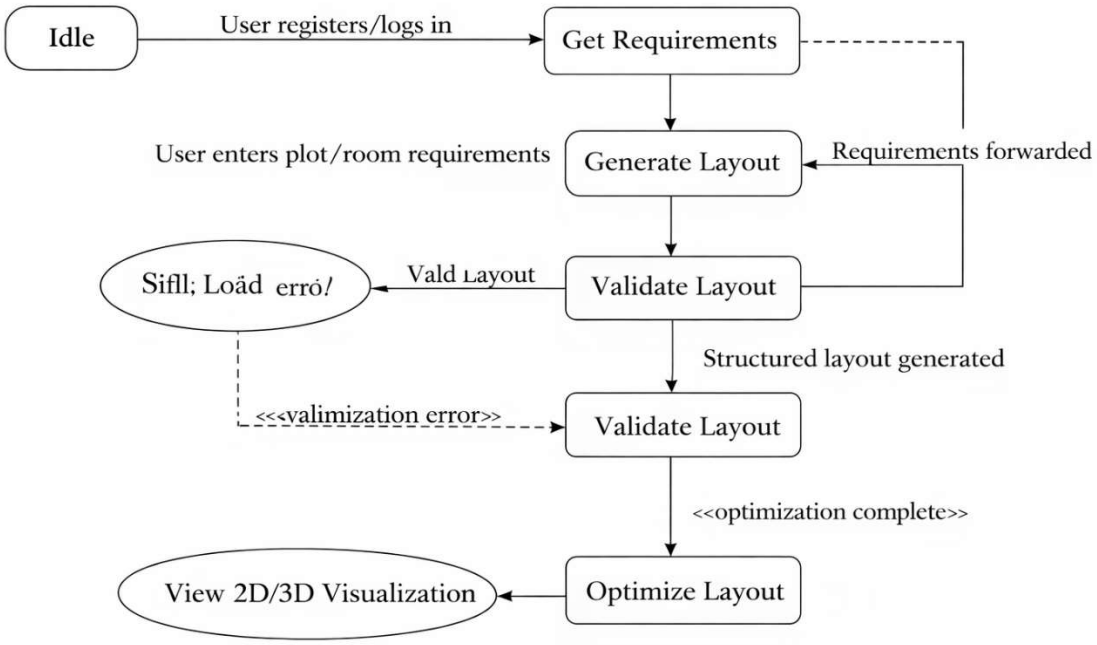
3.1 ARCHITECTURAL DESIGN



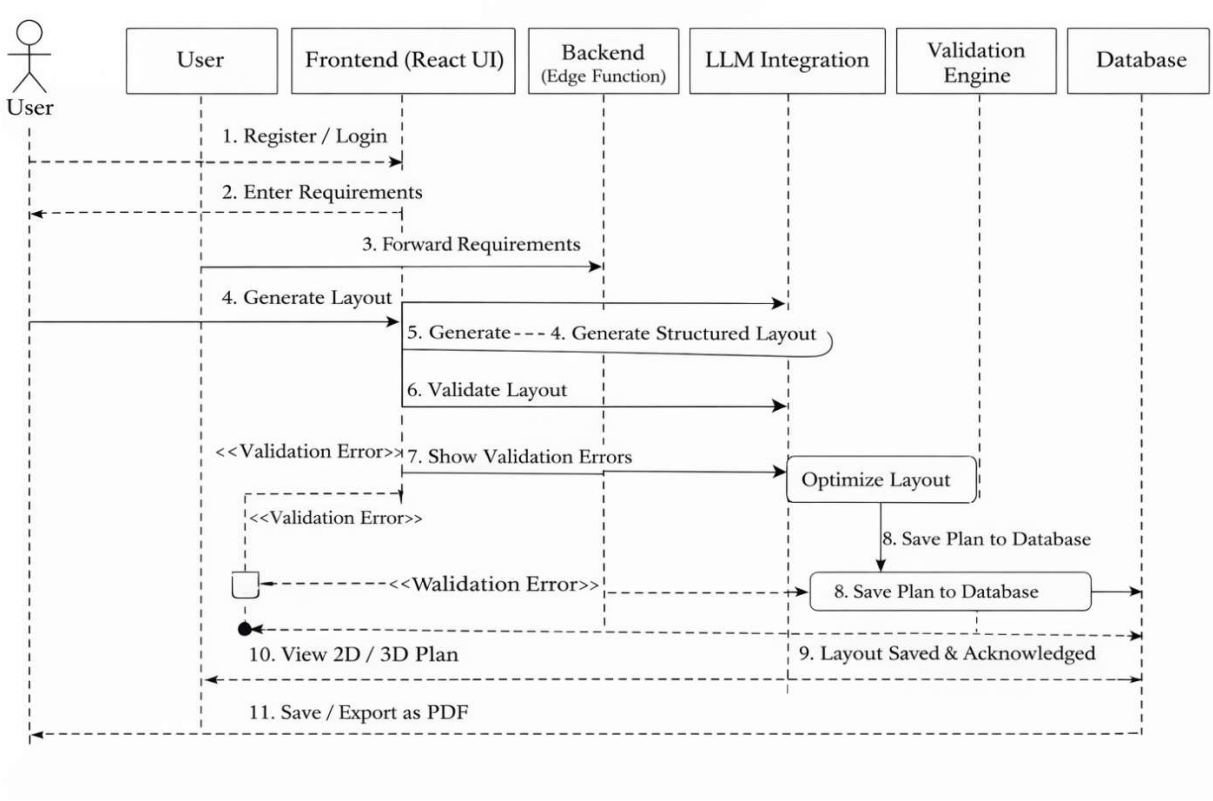
3.2 USECASE DIAGRAM



3.3 STATE DIAGRAM



3.4 SEQUENCE DIAGRAM



3.5 DESIGN PROCESS

The design process of the proposed system PlanAI follows a structured and modular software engineering approach to ensure clarity, maintainability, scalability, and performance efficiency. The system design is developed by dividing the overall workflow into logical components, each responsible for a specific function within the building plan generation pipeline. The design process begins with requirement analysis and system modeling. User requirements are first studied and categorized into functional and non-functional requirements. Functional requirements include layout generation, validation, optimization, visualization, storage, and export. Non-functional requirements include performance efficiency, scalability, usability, reliability, and security. Following requirement analysis, the system is divided into independent but interconnected modules.

These modules include the user interface module, requirement processing module, LLM integration module, validation engine, optimization module, visualization module, and data storage module. Each module is designed to operate independently while communicating through defined interfaces. This modular design ensures loose coupling and allows future modifications without affecting the entire system architecture. The system design also incorporates layered architecture principles. The presentation layer handles user interaction and visualization. The processing layer manages LLM invocation, rule validation, and optimization logic. The data and integration layer ensures secure storage, authentication, and external API communication. This layered approach improves separation of concerns and enhances maintainability.

Error handling and validation are integrated at multiple stages of the design. Input validation prevents incorrect data entry at the frontend. Backend validation ensures architectural compliance. Exception handling mechanisms are included to manage LLM response inconsistencies or validation failures. The design process also emphasizes scalability. Since layout generation involves computational processing and API communication, serverless backend functions are used to allow dynamic scaling based on user demand. Database schema design ensures efficient storage and retrieval of layout data. Finally, usability is considered during interface design. A guided multi-step input form simplifies requirement entry, and visual outputs are designed for clarity and interpretability. Both 2D blueprint rendering and 3D visualization are integrated to support technical and non-technical users.

3.5.1 DATABASE DESIGN

The database design of the proposed system PlanAI is structured to efficiently store user information, generated floor plans, layout metadata, and system-related records. The database follows a relational structure with clearly defined entities and relationships to ensure data consistency, security, and scalability. Since the system is cloud-based, a managed database service such as Supabase or a PostgreSQL-compatible backend is used to handle persistent storage. The primary entity in the database is the User table. This table stores user-related information such as unique user identifier, name, email address, encrypted authentication credentials, and account creation timestamp. Each user is uniquely identified by a primary key, ensuring secure association between users and their generated plans.

The second major entity is the Saved_Plan table. This table stores all generated building plans associated with users. It includes attributes such as plan ID, user ID (foreign key), plot dimensions, number of floors, number of rooms, layout JSON data, efficiency score, and creation timestamp. The structured layout JSON field stores room coordinates, dimensions, adjacency data, and floor mapping information generated by the LLM. The foreign key relationship between User and Saved_Plan ensures that each plan is linked to a specific authenticated user. Additional optional tables may include Plan_Metadata and System_Logs. The Plan_Metadata table can store computed metrics such as total built-up area, space utilization percentage, and optimization score.

The System_Logs table can record generation requests, validation errors, and API response logs for debugging and monitoring purposes. The database design ensures normalization to avoid redundancy. User details are stored separately from plan details, and each plan record references its owner through a foreign key. Indexing mechanisms are applied on frequently queried fields such as user ID and plan ID to improve retrieval speed. Security considerations are incorporated into the database design. Authentication tokens and encrypted credentials ensure that users can only access their own saved plans. Role-based access control can be implemented to restrict administrative operations.

3.5.2 INPUT DESIGN

The input design of the proposed system PlanAI focuses on creating a structured, user-friendly, and error-resistant interface for collecting building planning requirements. Since the system relies on accurate input parameters to generate optimized floor plans, careful attention is given to input validation, usability, and logical organization. The input interface is implemented as a guided multi-step form to simplify the requirement entry process. Instead of presenting all fields at once, the system divides inputs into logical sections such as plot details, building configuration, room requirements, and design preferences. This structured approach reduces cognitive load and improves user experience.

The primary inputs include plot length and width, number of floors, number of bedrooms, bathrooms, and additional spaces such as kitchen, living room, dining area, staircase, and utility rooms. Users may also select optional preferences such as layout style, orientation constraints, and

zoning preferences. All numerical inputs are validated for acceptable ranges to prevent unrealistic values. For example, plot dimensions must be positive numbers within predefined limits. Input validation is implemented at both client-side and server-side levels. Client-side validation ensures immediate feedback for incorrect entries such as empty fields, invalid characters, or out-of-range values. Server-side validation ensures data integrity before invoking the LLM generation process. This dual-layer validation minimizes system errors and prevents invalid layout generation requests.

Dropdown menus, numeric fields, and controlled selection inputs are used instead of free-text fields wherever possible. This structured input design ensures consistency and simplifies backend processing. Clear labels and tooltips provide guidance to users who may not be familiar with architectural terminology. The system also incorporates authentication-based input management. Only registered users can save generated plans, ensuring that input data is associated securely with the correct user account.

3.5.3 OUTPUT DESIGN

The output design of the proposed system PlanAI is structured to present generated building floor plans in a clear, professional, and technically interpretable format. Since the system produces structured layout data through LLM integration, the output module is responsible for transforming this data into visual and exportable representations that are useful for both technical and non-technical users. The primary output of the system is a 2D blueprint-style floor plan. This output is rendered using scalable vector graphics to ensure clarity and precision.

The 2D layout includes room boundaries, labeled room names, proportional dimensions, wall thickness differentiation, and spatial organization consistent with architectural standards. The design ensures that exterior and interior walls are visually distinguishable. Room segmentation is accurately represented based on the structured coordinate data generated by the backend. In addition to the 2D representation, the system provides an interactive 3D visualization of the building layout. This output is generated using browser-based 3D rendering technology.

Walls, floors, and openings are converted into three-dimensional models using spatial coordinates from the validated layout data. Users can rotate, zoom, and navigate the model to better understand spatial relationships and depth. This feature enhances user interpretation and supports better design decision-making. The system also supports downloadable output in PDF format. The PDF export includes the blueprint-style 2D layout with clear labeling and structured formatting suitable for sharing with contractors or architects. This ensures that the output is not limited to on-screen viewing but can be used for further professional refinement. Additional output data may include calculated metrics such as total built-up area, space utilization percentage, and layout efficiency score. These values provide quantitative insight into the quality and optimization of the generated plan.

3.6 TECHNIQUES USED

The proposed system PlanAI integrates multiple computational, architectural, and software engineering techniques to ensure accurate, efficient, and scalable building plan generation. These techniques combine intelligent layout synthesis with deterministic validation and optimization processes to deliver reliable outputs. One of the primary techniques used in the system is LLM-based structured generation. A Large Language Model is integrated through API communication and guided using structured prompting techniques. Instead of generating unstructured text, the model is instructed to produce layout data in a predefined JSON schema format. This technique ensures that room coordinates, dimensions, and adjacency information are returned in a machine-readable structure suitable for validation and rendering.

Another important technique is constraint-based validation. After layout generation, rule-based algorithms evaluate the output against architectural constraints such as minimum room dimensions, adjacency logic, circulation space requirements, and boundary alignment. This deterministic validation layer ensures that generated layouts comply with practical planning standards. The system also applies space optimization algorithms. These algorithms compute layout efficiency metrics such as space utilization ratio and detect unused or fragmented areas. Gap detection and spatial adjustment techniques are used to improve zoning balance and maximize functional coverage of the plot area.

A layered architectural design technique is employed to separate the presentation layer, processing layer, and data layer. This separation of concerns improves modularity and maintainability. Each module communicates through defined interfaces, reducing coupling and enabling future upgrades. For visualization, the system uses vector-based rendering techniques for 2D blueprint generation and real-time 3D rendering techniques for interactive visualization. The conversion of structured coordinate data into graphical representations follows geometric transformation principles to ensure dimensional accuracy. The system also implements client-side and server-side validation techniques to prevent invalid input processing. This dual-layer validation approach improves reliability and reduces runtime errors. Finally, the platform utilizes serverless cloud computing techniques for backend execution. Serverless functions dynamically scale based on user demand, ensuring efficient resource utilization and consistent performance.

IV. TESTING AND IMPLEMENTATION

4.1 SYSTEM TESTING

System testing is performed to ensure that the proposed system PlanAI functions correctly, efficiently, and reliably according to the defined requirements. Testing is carried out at multiple levels to validate functionality, performance, usability, and integration of all system components. Initially, unit testing is conducted on individual modules such as input validation, LLM invocation, rule-based validation engine, optimization module, and visualization components. Each module is tested independently to verify that it produces expected outputs under normal and boundary conditions. For example, input validation is tested with valid and invalid plot dimensions to ensure proper error handling.

Integration testing is performed to verify communication between frontend, backend, LLM integration layer, and database. The system is tested to ensure that user input is correctly transmitted to backend services, processed by the LLM, validated, optimized, and returned for visualization without data corruption or loss. Functional testing ensures that all core features operate according to system specifications. This includes testing layout generation, validation logic, multi-floor handling, plan saving, retrieval, and PDF export functionality. Each functional requirement is verified against expected outcomes.

Performance testing evaluates system response time and scalability. The layout generation process is tested under multiple concurrent requests to ensure that serverless backend functions handle load efficiently without significant delay. Rendering performance for 2D and 3D visualization is also evaluated. Usability testing is conducted to confirm that the user interface is intuitive and easy to navigate. Users are observed entering requirements, generating plans, and interacting with visual outputs to ensure clarity and simplicity of design. Error handling and exception testing are performed to evaluate system stability in cases such as invalid API responses, network interruptions, or incomplete user input. The system is tested to ensure graceful error messages and recovery mechanisms.

4.2 SYSTEM IMPLEMENTATION

System implementation refers to the practical realization of the proposed system PlanAI based on the designed architecture and defined specifications. This phase involves coding, integration of modules, deployment, and configuration of the application in a working environment. The implementation begins with frontend development using React and TypeScript to build the user interface components. The multi-step input form, navigation system, state management logic, and visualization components are developed and integrated. Client-side validation mechanisms are implemented to ensure that user inputs meet predefined constraints before being transmitted to the backend.

The backend implementation involves configuring cloud-based services such as database management, authentication mechanisms, and serverless edge functions. Edge functions are implemented to handle request processing, LLM invocation, structured response parsing, validation, and optimization logic. Secure API communication is established between the backend and the LLM service to ensure reliable generation of structured layout data. Database schema implementation includes creating tables for user accounts and saved plans, defining primary and foreign key relationships, and configuring indexing for efficient retrieval. Authentication mechanisms are implemented to ensure that users can securely store and access only their own generated plans.

Visualization modules are implemented to transform structured layout data into 2D blueprint-style representations and interactive 3D models. Rendering logic converts coordinate data into graphical elements while maintaining proportional accuracy. After development, the system is deployed on a cloud hosting environment. Environment variables are configured for secure API access, and HTTPS protocols are enabled to ensure secure communication. The application is then made accessible through a web browser without requiring local installation.

4.3 SYSTEM MAINTENANCE

System maintenance ensures that the proposed system PlanAI continues to function efficiently, securely, and reliably after deployment. Maintenance activities are essential to handle updates, bug fixes, performance improvements, and future feature enhancements. Corrective maintenance involves identifying and resolving software defects that may arise during system usage. This includes fixing frontend display issues, resolving backend processing errors, handling unexpected LLM responses, and correcting database inconsistencies. Monitoring tools and logs are used to detect runtime exceptions and performance anomalies.

Adaptive maintenance focuses on modifying the system to accommodate changes in external environments. This may include updating API configurations if the LLM provider modifies endpoints, upgrading dependencies such as frontend libraries, or adapting to updated browser standards and cloud platform changes. Perfective maintenance involves improving system performance and usability based on user feedback. Enhancements may include refining input forms, improving layout generation prompts, optimizing visualization rendering speed, or enhancing validation accuracy.

Preventive maintenance ensures long-term reliability by updating security patches, renewing authentication tokens, backing up database records, and improving code modularity. Regular database backups and cloud monitoring are implemented to prevent data loss and downtime. Scalability monitoring is also part of maintenance. As user demand increases, serverless configurations and database performance settings may be adjusted to maintain consistent response times.

V. CONCLUSION

The proposed system PlanAI demonstrates the effective integration of Large Language Model based generation, rule-driven validation, spatial optimization, and modern web technologies to automate the conceptual building floor plan design process. Traditional architectural planning methods rely heavily on manual drafting, iterative revisions, and professional intervention, which increase time and cost during the early design stage. The developed system addresses these challenges by introducing an intelligent, requirement-driven workflow that converts structured user inputs into optimized and validated building layouts.

Through structured layout generation, deterministic validation mechanisms, and computational space optimization, the system ensures that generated floor plans are not only produced quickly but also maintain architectural feasibility and logical spatial organization. The inclusion of dual visualization modes, including 2D blueprint representation and interactive 3D rendering, enhances usability for both technical and non-technical users. The cloud-based architecture and serverless processing model provide scalability and accessibility, allowing multiple users to generate and manage floor plans without requiring specialized hardware or software installations. The modular design ensures maintainability and future extensibility.

VI. SCOPE OF THE FUTURE DEVELOPMENT

The proposed system PlanAI establishes a strong foundation for AI-assisted architectural layout generation, and there are several opportunities for future enhancements to expand its capabilities and practical applicability. One major area of future development is the integration of structural and engineering analysis modules. The current system focuses on conceptual layout generation; however, future versions can incorporate structural load calculations, foundation recommendations, and material strength analysis to move closer to construction-ready documentation. Another potential enhancement is the integration of electrical, plumbing, and HVAC layout planning. By extending the structured layout data, the system can automatically suggest wiring routes, plumbing lines, and ventilation ducts, thereby supporting more comprehensive building design automation. Cost estimation and material recommendation modules can also be integrated.

Based on the generated layout dimensions and area calculations, the system could estimate approximate construction costs and suggest suitable building materials according to budget constraints. Future development may include direct CAD file export functionality such as DXF or DWG formats. This would allow seamless integration with professional architectural software for further refinement and approval processes. The system can also incorporate advanced AI fine-tuning using domain-specific datasets to improve architectural accuracy and regional adaptability. Local building regulations and municipal compliance checks could be integrated to enhance legal feasibility. Collaborative design features can be introduced to allow multiple users, architects, or stakeholders to review and modify layouts in real time. Integration with augmented reality or virtual reality technologies could further enhance spatial visualization and user engagement. Additionally, performance optimization and mobile responsiveness improvements can expand accessibility across various devices.

BIBLIOGRAPHY

1. Nauata, N., Chang, K., Cheng, C., Mori, G., and Furukawa, Y., "House-GAN: Relational Generative Adversarial Networks for Graph-Constrained House Layout Generation," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021.
2. Nauata, N., et al., "House-GAN++: Generative Adversarial Layout Refinement Network Towards Intelligent Computational Agent for Floor Plan Design," IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2021.
3. Liu, C., Wu, J., and Furukawa, Y., "FloorNet: A Unified Framework for Floorplan Reconstruction from 3D Scans," European Conference on Computer Vision (ECCV), 2018.
4. Merrell, P., Schkufza, E., Li, Z., Agrawala, M., and Koltun, V., "Computer-Generated Residential Building Layouts," ACM Transactions on Graphics (SIGGRAPH), 2010.
5. Wang, R., Li, X., and Zhang, Y., "Automated Building Layout Generation Using Deep Learning and Graph Algorithms," Automation in Construction, Elsevier, 2023.
6. React Documentation, "React – A JavaScript Library for Building User Interfaces," Meta Open Source Documentation, https://react.dev
7. TypeScript Documentation, "TypeScript: Typed JavaScript at Any Scale," Microsoft Documentation, https://www.typescriptlang.org
8. Three.js Documentation, "Three.js – JavaScript 3D Library," https://threejs.org
9. Supabase Documentation, "Supabase: Open Source Backend Platform," https://supabase.com/docs
10. Google AI Documentation, "Large Language Models and Generative AI Systems," Google AI Technical Resources.

APPENDICES

(A) SOURCE CODE:

```
import { useState } from "react";

export default function PlanForm() {
  const [formData, setFormData] = useState({
    plotLength: "",
    plotWidth: "",
    bedrooms: 0,
    bathrooms: 0,
    floors: 1,
  });
  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value,
    });
  };
  const handleSubmit = async () => {
    const response = await fetch("/api/generate-plan", {
```

```
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(formData),
  });
  const result = await response.json();
  console.log(result);
};
return (
  <div>
    <input name="plotLength" placeholder="Plot Length"
      onChange={handleChange} />
    <input name="plotWidth" placeholder="Plot Width"
      onChange={handleChange} />
    <input name="bedrooms" type="number"
      onChange={handleChange} />
    <input name="bathrooms" type="number"
      onChange={handleChange} />
    <button onClick={handleSubmit}>
      Generate Plan
```

```
      </button>
    </div>
  );
}
import { serve } from "https://deno.land/std/http/server.ts";
serve(async (req) => {
  const body = await req.json();
  const prompt = `
  Generate a building layout in JSON format.
  Plot: ${body.plotLength} x ${body.plotWidth}
  Bedrooms: ${body.bedrooms}
  Bathrooms: ${body.bathrooms}
  Floors: ${body.floors}
  Return structured JSON with room coordinates.
  `;
      "Authorization": `Bearer ${Deno.env.get("LLM_API_KEY")}`,
    },
    body: JSON.stringify({ prompt }),
  });
```

(B) SCREEN SHOTS:

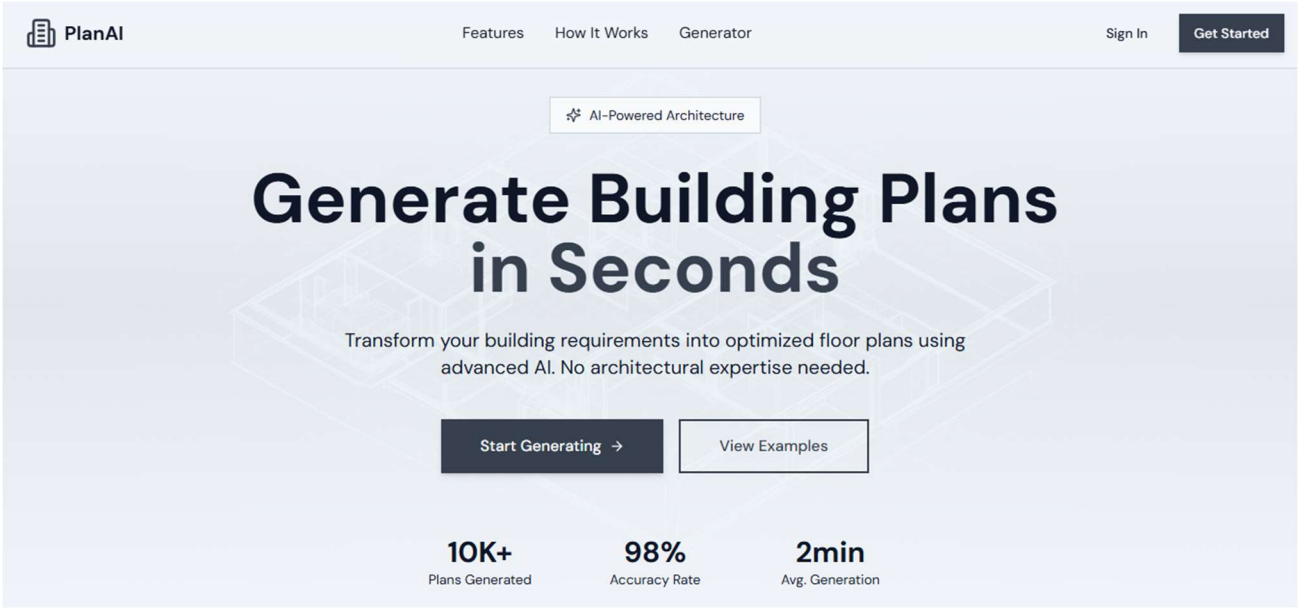


Fig.1. Proposed System

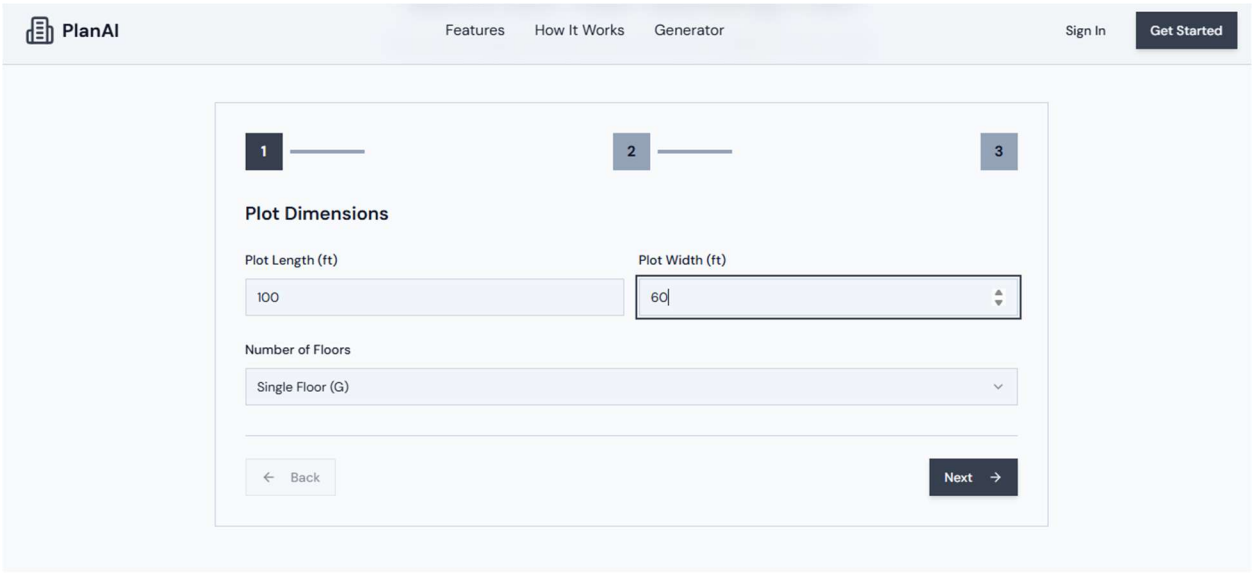


Fig.2. Dimension Parameters

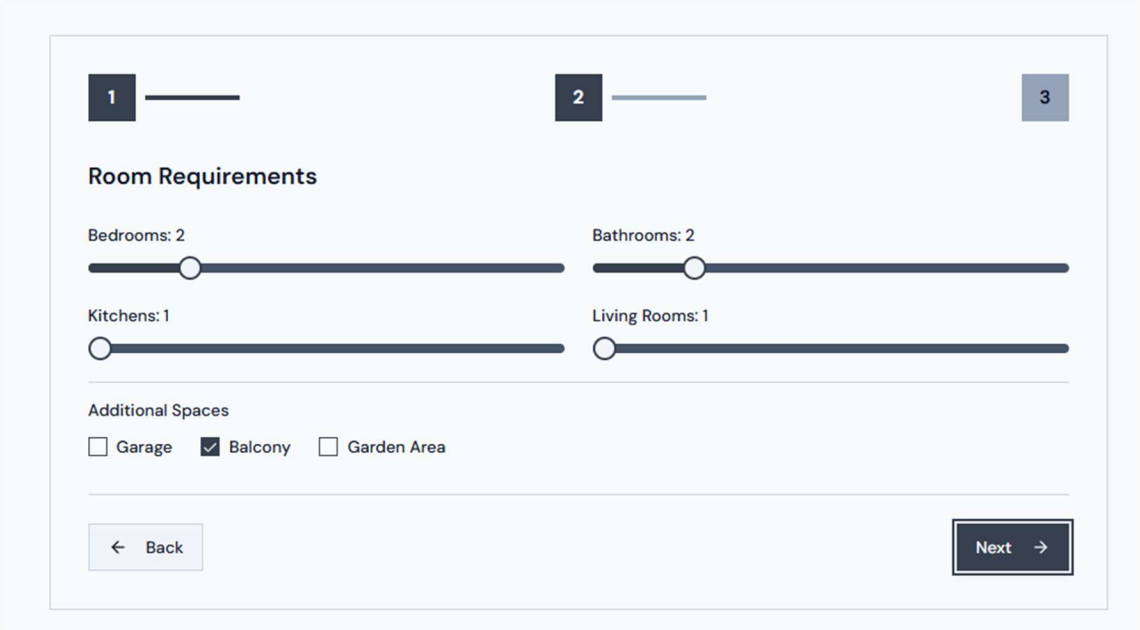


Fig.3. Room Parameters

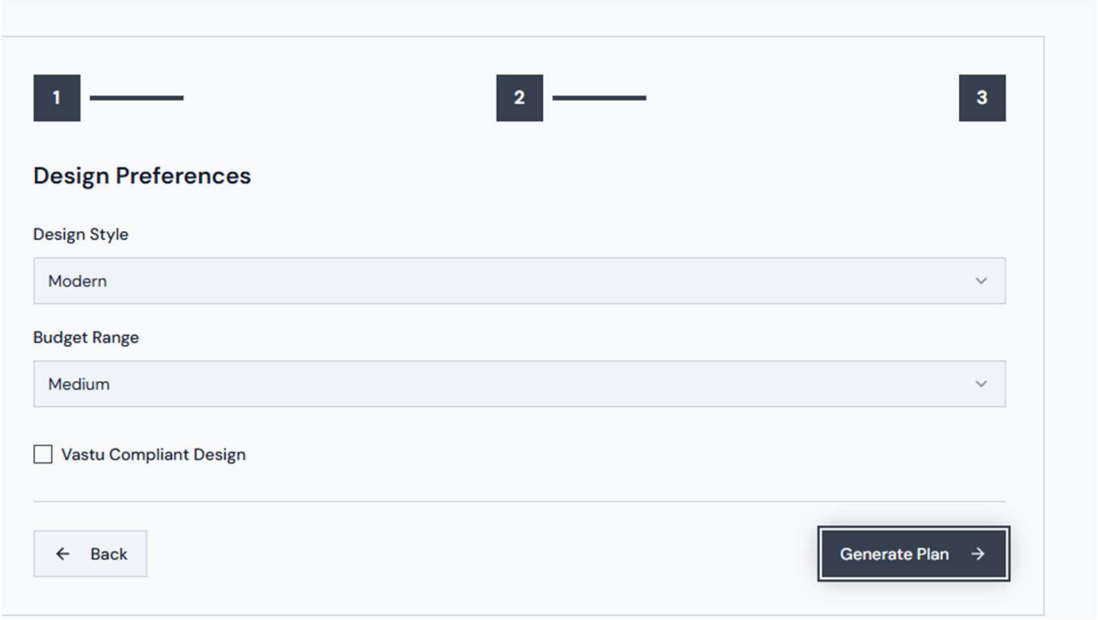


Fig.4. Design Parameters

(C) TABLES:

Table 1: Plot and Building Parameters

Parameter Name	Type	Description
plot_length	Numeric	Length of the plot in feet or meters
plot_width	Numeric	Width of the plot in feet or meters
floors	Integer	Number of building floors
building_type	String	Residential or Commercial type
orientation	String	Directional preference (North, East, etc.)

Table 2: Room Requirement Parameters

Parameter Name	Type	Description
bedrooms	Integer	Number of bedrooms required
bathrooms	Integer	Number of bathrooms required
kitchen	Boolean	Whether kitchen is required
living_room	Boolean	Whether living room is required
dining_room	Boolean	Whether dining area is required
staircase	Boolean	Whether staircase is required
utility_room	Boolean	Optional utility space

Table 3: Constraint Parameters

Constraint Name	Type	Description
minimum_room_size	Numeric	Minimum allowable room dimension
circulation_width	Numeric	Minimum passage width
adjacency_rules	String	Room connectivity preferences
ventilation_requirement	Boolean	Enforces window placement rules
zoning_preference	String	Public and private zone separation

Table 4: LLM Output Structure

Field Name	Type	Description
room_id	String	Unique identifier for each room
room_type	String	Type of room (Bedroom, Kitchen, etc.)
x_coordinate	Numeric	Starting X position of the room
y_coordinate	Numeric	Starting Y position of the room
width	Numeric	Room width
height	Numeric	Room height
floor_index	Integer	Floor number
adjacency_list	Array	Connected rooms

(D) REPORTS:

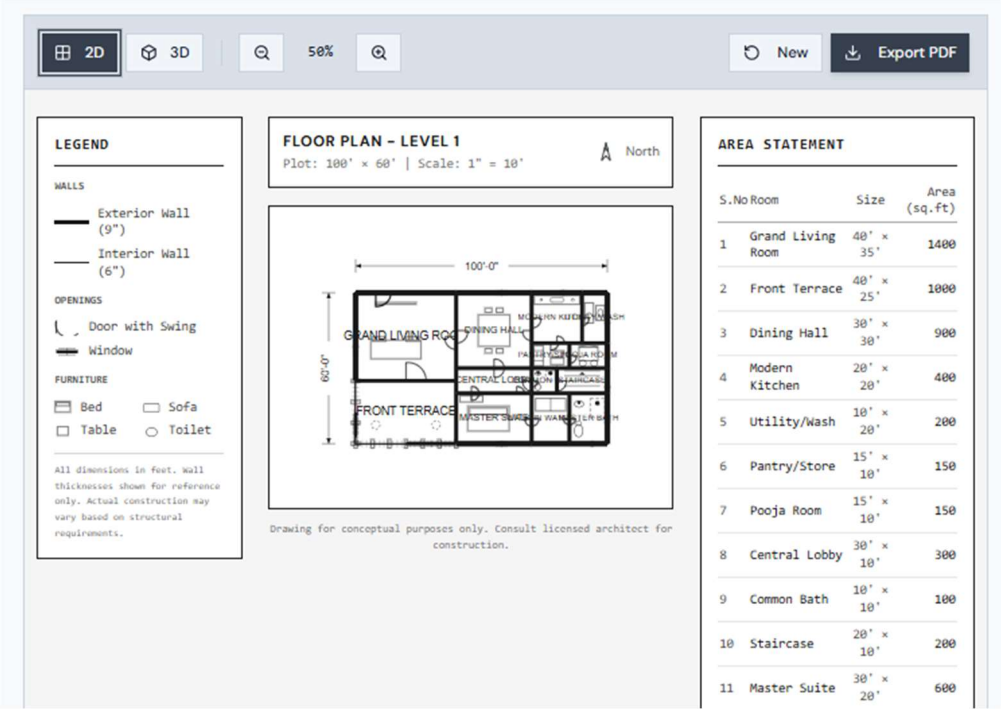


Fig.5. 2D Output Building Plan



Fig.6. 3D Output Building Plan